

ML Ops

Major Assignment

End-to-End MLOps Pipeline

Roll No	G25AI1016
Name	Vijay Sai Krishna D
Program	PG Diploma in AI & ML (MLOps)
Batch	May 2026
Date of Submission	June 2026

GitHub Repository	https://github.com/Vijaysaii/MLOps-Assignment-Major-g25ai1016
Docker Hub Repository	https://hub.docker.com/r/vijaysaii/olivetti-faces-app

1. Overview

This assignment focuses on building a complete, end-to-end MLOps pipeline. The goal was to train a machine learning model using scikit-learn, automate the training and testing process using GitHub Actions CI/CD, containerize the application using Docker, and deploy it on Kubernetes with high availability using 3 replicas.

The Olivetti Faces dataset from sklearn.datasets was used to train a DecisionTreeClassifier model. The entire workflow follows a structured Git branching strategy with three branches: main, dev, and docker_cicd.

2. Dataset & Model

2.1 Dataset

The Olivetti Faces dataset is a classic face recognition dataset available in scikit-learn. It contains 400 grayscale face images of 40 different subjects, with 10 images per subject. Each image is 64x64 pixels, giving a feature vector of 4096 values per sample.

Property	Value
Total Samples	400
Number of Classes	40 (subjects)
Image Size	64 x 64 pixels
Feature Vector Size	4096
Train Set (70%)	280 samples
Test Set (30%)	120 samples

2.2 Model

A DecisionTreeClassifier from scikit-learn was used as the classification model. The model was trained on the 70% training split and evaluated on the 30% test split. The trained model was serialized using joblib and saved as savedmodel.pth for reuse in the Flask inference application.

The model achieved a test accuracy of **55.00%** on the 120-sample test set.

3. Git Branching Strategy

The project follows a strict three-branch Git strategy as required by the assignment. Each branch has a specific purpose and the branches are not merged back into main.

Branch	Purpose	Key Files
--------	---------	-----------

main	Initial project setup	README.md, .gitignore
dev	Model development + CI/CD	train.py, test.py, requirements.txt, ci.yml
docker_cicd	Docker + Kubernetes deployment	app.py, Dockerfile, k8s/deployment.yaml, k8s/service.yaml

4. Step-by-Step Implementation

4.1 Step 1 - main Branch: Initial Setup

The main branch was initialized first with a README.md and a .gitignore file. The README contains a full description of the project, repository structure, branch strategy, and usage instructions. The .gitignore was configured to exclude Python cache files, virtual environments, the saved model file, and IDE files.

- Initialized local Git repository with: git init
- Created main branch: git checkout -b main
- Committed README.md and .gitignore to main
- Pushed main branch to GitHub

4.2 Step 2 - dev Branch: Model Development

A new branch named dev was created from main. This branch contains all the model training, evaluation, and CI/CD pipeline code.

train.py

This script loads the Olivetti Faces dataset, splits it 70/30, trains a DecisionTreeClassifier, and saves the model along with the test data using joblib.

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
import joblib

data = fetch_olivetti_faces(shuffle=True, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.30, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
joblib.dump((model, X_test, y_test), 'savedmodel.pth')
```

test.py

This script loads the saved model and test data, runs predictions, and prints the test accuracy.

```
import joblib
from sklearn.metrics import accuracy_score

model, X_test, y_test = joblib.load('savedmodel.pth')
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Test Accuracy: {accuracy * 100:.2f}%')
```

CI/CD Workflow (.github/workflows/ci.yml)

The GitHub Actions workflow runs on every push to the dev and docker_cicd branches. It has one job called check_working_repo which performs checkout, Python setup, dependency installation, model training, and accuracy evaluation.

```
name: CI Pipeline
on:
  push:
    branches: [dev, docker_cicd]

jobs:
  check_working_repo:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements.txt
      - run: python train.py
      - run: python test.py
```

4.3 Step 3 - docker_cicd Branch: Docker and Kubernetes

The docker_cicd branch was created from dev. It adds a Flask web application, a Dockerfile, and Kubernetes manifests for deployment.

Flask Web Application (app.py)

The Flask app provides a simple web interface where a user can upload a 64x64 grayscale face image. The image is preprocessed, passed through the trained model, and the predicted subject ID (0-39) is displayed on the page.

- Route: GET / - Shows the upload form
- Route: POST / - Accepts image, runs prediction, shows result
- Image is converted to grayscale, resized to 64x64, and normalized

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY train.py test.py app.py ./
RUN python train.py
EXPOSE 5000
CMD ["python", "app.py"]
```

The Dockerfile uses a slim Python 3.11 base image, installs all dependencies, copies the source files, trains the model at build time, and starts the Flask server on port 5000.

Building and Pushing Docker Image

```
docker build -t vijaysaii/olivetti-faces-app:latest .  
docker push vijaysaii/olivetti-faces-app:latest
```

5. Kubernetes Deployment

The application was deployed on a local Kubernetes cluster using minikube. A Deployment with 3 replicas ensures high availability and self-healing. A NodePort Service exposes the app on port 30080.

5.1 Deployment Manifest (k8s/deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: olivetti-faces-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: olivetti-faces-app
  template:
    spec:
      containers:
        - name: olivetti-faces-app
          image: vijaysaii/olivetti-faces-app:latest
          ports:
            - containerPort: 5000
```

5.2 Service Manifest (k8s/service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: olivetti-faces-service
spec:
  type: NodePort
  selector:
    app: olivetti-faces-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
```

5.3 Deploying and Verifying

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods
minikube service olivetti-faces-service
```

Three pods were verified to be running after the deployment. To demonstrate self-healing, one pod was deleted using `kubectl delete pod` and it was confirmed that Kubernetes automatically started a new pod to maintain the 3-replica count.

```
kubectl delete pod olivetti-faces-app-  
kubectl get pods --watch    # shows new pod being created
```

6. Analysis & Observations

6.1 Model Performance

The DecisionTreeClassifier achieved a test accuracy of 55.00% on the Olivetti Faces dataset with a 70/30 train-test split. This is a reasonable baseline for a Decision Tree on face recognition since Decision Trees do not capture spatial features as effectively as CNNs. The model overfits the training data slightly due to the high-dimensional input (4096 features) and the small dataset size. Performance could be improved using Random Forests, SVMs, or deep learning models, but the goal of this assignment was to demonstrate the MLOps workflow rather than achieve maximum accuracy.

6.2 CI/CD Pipeline

The GitHub Actions workflow successfully ran on the dev branch on every push. It automated the full ML lifecycle - from installing dependencies to training the model and printing evaluation metrics - without any manual intervention. This is the core principle of MLOps: making ML workflows reproducible and automated.

6.3 Docker Containerization

Containerizing the app ensures the model runs in the same environment regardless of the host machine. The model is trained inside the Docker image at build time, so the container is fully self-contained. This eliminates dependency issues and makes the app portable across cloud platforms.

6.4 Kubernetes Orchestration

Running 3 replicas with a Deployment ensures that if one pod crashes, Kubernetes automatically replaces it. This was demonstrated by deleting a pod and observing a new one being scheduled immediately. The NodePort service makes the app accessible on a fixed port without needing a cloud load balancer.

7. Conclusion

This assignment gave hands-on experience with a real MLOps pipeline from start to finish. I learned how to structure a machine learning project with proper Git branching, automate training and testing with GitHub Actions, package a model into a Docker container, and deploy it on Kubernetes with fault tolerance.

The most interesting part was seeing how all the pieces work together - a code push triggers the CI pipeline, the model gets trained automatically, and the same model is then served through a

containerized Flask API that Kubernetes keeps running at all times. This is exactly how production ML systems work.

8. Repository Links

GitHub Repository	https://github.com/Vijaysaii/MLops-Assignment-Major-g25ai1016
Docker Hub	https://hub.docker.com/r/vijaysaii/olivetti-faces-app